Identifying LLM Authors from Text

Comparing CNN and LSTM Classifiers
CMPSC 448 Midterm Project
Aidan Brown (solo group)
October 11, 2026

Problem Definition

For my project, I decided to investigate whether plain text has enough information to identify the specific LLM that generated the text. It is three-class classification: GPT-4, LLaMA-2-13B-chat, or Falcon-40B-instruct. Each record contains a generator label, a user instruction (prompt), and the response given by the LLM. The two research questions I am focusing on is whether the generator can be identified from its response and whether the user prompt helps identification. Two required neural architectures were implemented (CNN and LSTM). Each was evaluated first with only prompts, then only response, then combined prompt and response.

Dataset and Curation

The data was originally from UltraFeedback dataset by OpenBMB. The dataset contains 64,000 instructions and 256,000 responses from multiple LLMs. For this project, I used existing responses instead of asking questions to LLMs. The code scanned 63,967 prompts and the results came out with 16,518 GPT-4 responses, 16,632 LLaMA responses, and 16,487 Falcon responses. Reservoir sampling retained 1,000 examples per model while scanning the dataset. I excluded empty instructions and responses as well as repeated records of prompts or responses to reduce duplicate-response leakage. The remaining classes were made to be equal size with 977 examples per model and 2,931 total examples. The field names include LLM_name, LLM_Input, and LLM_Output. LLM_Input is the prompt that the user gives to the LLM (just the prompt not the full chat). The dataset card lists an MIT license, upstream source terms, and personal-information review. The prompts were normalized to avoid complications with capital and lowercase letters with whitespace as space and then hashed to define each group. I split the data 70% of records to training, 15% to validation, and 15% to testing. The responses that shared a normalized user prompt were in one split. There was zero overlap between splits, however, there was a little imbalance between splits.

Split	GPT-4	LLaMA	Falcon	Total
Training	681	717	657	2055
Validation	154	129	154	437
Test	142	131	166	439
Overall	977	977	977	2931

Implementation and Training

I made the text lowercased and tokenized it into words and punctuation using a regular expression. For each input setting, a vocabulary was fitted only on the training portion, capping at 15,000 entries. These entries include padding, unknown-token, and separator IDs. Prompt-only and response-only settings used the beginning of the relevant text. The combined input retained a max of 128 prompt tokens, a separator, and the rest were response tokens.

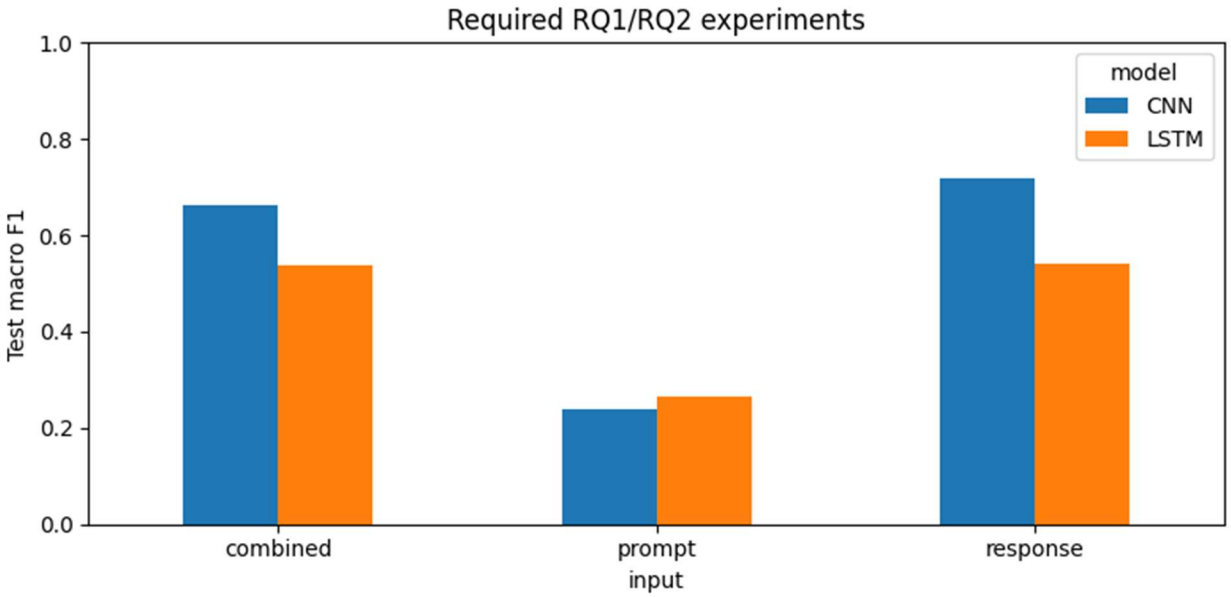
CNN: The CNN converted each text token into a numerical representation then looked for patterns in token groups of 3 to 5. It kept the strongest pattern recognitions and combined them to predict which LLM the text was generated by.

LSTM: The LSTM also converted the tokens in a way like the CNN. However, it read them in order and maintained a memory of all the earlier tokens. It used its memory to predict which model generated the text. The padding added to the texts made them the same length.
Both models learned in batches of 32 examples using the Adam optimizer and a learning rate of 0.001. Dropout and gradient clipping reduced overfitting and kept the training stable. Each model can train for a max of 8 passes through the training data and stopped early if the macro F1 score did not improve from the last pass. Fixed random seeds were used to make the results reproducible.

Experimental Results

The accuracy category measures the percentage of correct classifications. The macro F1 score takes the average of F1 scores equally and it is useful when the test split isn't balanced exactly. All models used the same test records (439). The expected accuracy would be 33.33%.

Model	Input	Accuracy	Macro F1	Best epoch
CNN	Prompt	32.57%	0.238	3
LSTM	Prompt	30.30%	0.266	1
CNN	Response	70.84%	0.717	1
LSTM	Response	54.44%	0.542	5
CNN	Prompt and Response	64.92%	0.663	2
LSTM	Prompt and Response	54.67%	0.538	5

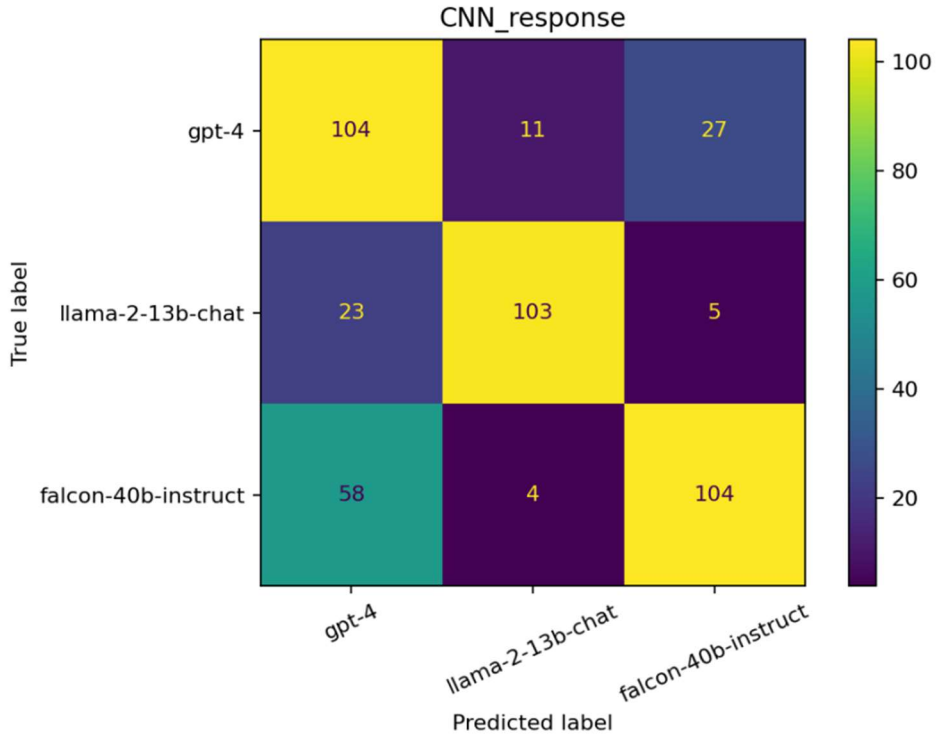


RQ1: Can the generator be identified from its response?
Both models performed better than random guessing (33.33% accuracy). Using only the response data, the CNN correctly identified the LLM 70.84% of the time while only 54.44% for LSTM. CNN performed better across the board apart from just the prompt where it performed slightly worse. It is worth noting that these findings only apply to the three models that were tested.

RQ2: Does including the prompt help?
Using only the prompt produced low accuracy for both models. The CNN had 32.57% and LSTM had 30.30% which are both lower than the expected accuracy. Adding the prompt lowered the CNN accuracy and helped the LSTM only one additional example correctly. Overall, including the prompt did not improve performance consistently.

Error Analysis and Interpretation

Generator	CNN response precision	Recall	F1	LSTM response F1
GPT-4	0.562	0.732	0.636	0.545
LLaMA	0.873	0.786	0.827	0.502
Falcon	0.765	0.627	0.689	0.577



The CNN was the best at identifying LLaMA responses and had the most trouble identifying GPT-4 responses. Its most common mistake was labeling other responses as GPT-4. It also missed various Falcon responses. The CNN may have performed better because it recognized useful patterns in short groups of tokens. The patterns CNN used, however, were not tested. Adding more prompts may have been harmful as it would have possibly introduced less useful information and left less space for a response. Both models showed signs of overfitting as they continued improving on the training data without improving on the validation data. I used the version with the best validation score to help with this. Some limitations may include the models learning differences in topics or prompting styles instead of the actual writing styles. Some responses could've specifically stated their own name despite cleaning. The lowercasing and shortening may have also removed information that would've been useful for these models. Also, these finding may not

translate to a more current LLM. The experiment also only used one random seed and one data split. Changing some of these variables may produce a different result.

Lessons and Experience

Through completing this project, I not only learned more about these LLMs, but I got hands on experience that helped me understand training and accuracy better. This project also showed how data splitting and multiple metrics matter. By grouping each example by prompt made the evaluation focus on unseen prompts. Analyzing the different variables and understanding why they were the number that they were was another thing that I feel I learned immensely. Some difficulties I experienced were coding through google colab which I am not very familiar with. Generating the graphs and results was also another difficulty that I had to do research on. The code itself was also difficult to write, and I ran into various difficulties getting it to work.

Conclusion and References

The response-only CNN had the highest accuracy at 70.84% and 0.717 macro F1. Both response-only models exceeded the expected accuracy. These conclusions only apply to this dataset, and further testing is required for other datasets.

Dataset: https://huggingface.co/datasets/openbmb/UltraFeedback

Reference Paper: https://arxiv.org/abs/2310.01377